"GAN + CNN Architecture for Estimating Beam Parameters using BPM images for Indus Particle Accelerator"

Project Report submitted at

Beam Diagnostic Section

GOVERNMENT OF INDIA

DEPARTMENT OF ATOMIC ENERGY

RAJA RAMANNA CENTRE FOR ADVANCED TECHNOLOGY

INDORE (M. P.), 452013

Submitted by:

Mr. Pranshu Upadhyay
BTech. Computer Science Engineering
Shiv Nadar University
Delhi NCR

Guided by:

Dr. Surendra Yadav
Scientific Officer-G
Beam Diagnostic Section
RRCAT, Indore

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide, Dr. Surendra Yadav, Scientific Officer 'G', Beam Diagnostics Section, Accelerator Controls & Beam Diagnostics Division, RRCAT, Indore, for his valuable guidance, continuous support, and encouragement throughout my project titled **"GAN + CNN Architecture for Estimating Beam Parameters using BPM images for Indus Particle Accelerator".** His deep knowledge, patience, and insightful suggestions were instrumental in the successful completion of this work. It has been a great honour and privilege to carry out this project under his supervision.

I would also like to extend my heartfelt thanks to Shri V.P. Bhanage, Outstanding Scientist and Head, Accelerator Controls & Beam Diagnostics Division, RRCAT, Indore, for his constant motivation and for providing me the opportunity to pursue this internship in such a reputed institute.

My special thanks to the Project Placement Committee of RRCAT for granting me this valuable opportunity to intern with the Beam Diagnostics Section.

I also sincerely thank Sh. Avnish Ojha for his technical support, guidance, and encouragement throughout the training period.

Lastly, I would like to acknowledge the support and cooperation of all the staff members of the Beam Diagnostics Section, whose valuable suggestions and assistance helped me during my project work.

Mr. Pranshu Upadhyay
BTech. Computer Science Engineering
Shiv Nadar University, Delhi NCR

Government of India
Department of Atomic Energy
RAJA RAMANNA CENTRE FOR ADVANCED TECHNOLOGY
Indore-452013, India

CERTIFICATE

This is to certify that the dissertation work entitled, "**Deep Learning Models for Parameters Estimation of Electron Beam using Screen Monitor Images for Indus Accelerator System**" submitted by **Pranshu Upadhyay,** student of Shiv Nadar University, Dehli NCR, India is hereby approved as a credible work carried out by her during the period of 59 days from 2nd June 2025 to 31st July 2025 at **Beam Diagnostics Section, Accelerator Controls & Beam Diagnostics Division, RRCAT, Indore**, under my supervision.

Head of Division

Shri V.P. Bhanage
Outstanding Scientist
Head, Accelerator Controls & Beam Diagnostics Division, RRCAT, Indore.

Project Guide

Dr. Surendra Yadav
Scientific Officer 'G'
Beam Diagnostics Section, Accelerator Controls & Beam Diagnostics Division, RRCAT, Indore.

DECLARATION

I, **Pranshu Upadhyay**, hereby declare that the work presented in this report entitled **"GAN + CNN Architecture for accurately measuring Beam Size, Position using BPM images for Indus Particle Accelerator"** is my original work carried out at **Raja Ramanna Centre for Advanced Technology (RRCAT), Indore**, under the supervision of **Dr. Surendra Yadav**, Scientific Officer 'G', Beam Diagnostics Section, RRCAT, Indore.

This work has not been submitted to any other institution or university for the award of any degree or diploma.

Mr. Pranshu Upadhyay
BTech. Computer Science Engineering
Shiv Nadar University, Delhi NCR

Abstract

In the modern era of scientific and industrial advancement, particle accelerators have become foundational tools for exploring fundamental physics, developing advanced materials, and enabling medical technologies. Facilities like INDUS-1 and INDUS-2 at the Raja Ramanna Centre for Advanced Technology (RRCAT) represent state-of-the-art accelerator technology in India. These machines generate and manipulate beams of high-energy electrons, producing synchrotron radiation essential for a variety of applications. However, with increasing complexity and tighter operational tolerances, the need for smart beam diagnostics and control systems has become more pressing.

Traditional diagnostic systems such as Beam Position Monitors (BPMs) provide real-time feedback on beam position, envelope, and stability, but they are limited by signal noise, resolution, and manual tuning. As the size of datasets grows and beam dynamics become more intricate, there's a compelling need to integrate Artificial Intelligence (AI) and Machine Learning (ML) methods into these systems. Deep Learning, a subset of ML, has shown transformative potential in fields like computer vision, speech recognition, and autonomous systems. This report explores its application in the domain of accelerator physics, particularly focusing on image-based BPM diagnostics.

We present a pipeline combining Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs) to:

1. Denoise and process BPM images
2. Predict beam parameters such as horizontal and vertical position (x, y), beam size (σx, σy) (and in a later stage rotation (θ))
3. Augment datasets via synthetic image generation
4. Lay the groundwork for a feedback-based beam stabilization system

Our work begins with the design of synthetic datasets that simulate various real-world conditions — from noise and blur to offset and tilt. A GAN architecture is used to generate labeled, high-quality images to supplement training data. A CNN architecture is trained on this diverse dataset to extract quantitative beam parameters directly from raw BPM images.

Through this research, we demonstrate that AI-ML techniques can significantly enhance beam diagnostics and potentially automate calibration and control of magnetic optics in real time, leading to increased beamline uptime, reduced manual tuning, and improved experiment reliability.

Table of Contents

S.No.	Topic	Page No.
	Acknowledgement	2
	Certificate	3
	Declaration	4
	Abstract	5
	Table of Contents	6-7
Chapter 1	**Introduction: Particle Accelerators, AI-ML and How it all comes together**	8-15
	1.1 RRCAT, Particle Accelerators	8
	1.2 INDUS 1 & 2	9-10
	1.2.1 INDUS 1	
	1.2.2 INDUS 2	
	1.3 AI-ML	
	1.3.1 CNN Models	10-13
	1.3.2 GAN Models	
	1.4 BPMs	13-14
	1.5 CNN + GAN Tech for BPM	14-15
Chapter 2	**System Architecture Planning (Project Pipeline)**	16-24
	2.1 Dataset	16-23
	2.1.1 Data Acquisition	
	2.1.2 Main Script for Generating Base Dataset	
	2.1.3 Data Cleaning	
	2.1.4 Data Analysis	
	2.1.5 Data Reproduction using GAN	
	2.2 CNN for BPM Predictions	24
Chapter 3	**GAN Architecture, Planning and Implementation**	25-
	3.1 GAN Structure Overview	25
	3.2 Model Summary	26
	3.3 GAN Python Implementation	27-28
	3.4 Training Details and Hyperparameters	28

	3.5 Evaluation Metrics **3.6** Output and Usage	28-30 30
Chapter 4	**CNN Architecture, Planning and Implementation** **4.1** CNN Structure Overview **4.2** Model Summary **4.3** CNN Python Implementation **4.4** Training Details and Hyperparameters **4.5** Evaluation Metrics **4.6** Output and Usage	31-38 31-32 32 33 34 34-37 37-38
Chapter 5	**Future Scope of the Project** **5.1** Improved CNN and GAN Architecture **5.1.1** CNN Enhancements **5.1.2** GAN Enhancements **5.2** Customised Graphical User Interface (GUI) **5.2.1** GUI Objectives **5.2.2** Technical Stack **5.3** Feedback-Based Smart Control System for Real-Time Monitoring and Calibration **5.3.1** System Integration **5.3.2** Challenges and Solutions **5.3.3** Potential Outcomes	39-42 39-40 40-41 41-42
	List of Figures **Bibliography**	43 44-45

Chapter 1:

Introduction — Particle Accelerators, AI-ML, and How It All Comes Together

1.1 RRCAT and Particle Accelerators

The **Raja Ramanna Centre for Advanced Technology (RRCAT)**, located in Indore, Madhya Pradesh, is a premier national research laboratory functioning under the aegis of the **Department of Atomic Energy (DAE), Government of India**. The centre was established in **1984** by the visionary physicist and former Chairman of the Atomic Energy Commission, **Dr. Raja Ramanna**, after whom the centre is named. The founding objective of RRCAT was to pursue cutting-edge research and development in the areas of **lasers, particle accelerators, cryogenics**, and related technologies, which have far-reaching applications in both fundamental research and industrial sectors.

RRCAT plays a vital role in strengthening India's self-reliance in accelerator-based science and technology. Over the years, it has evolved into a multidisciplinary institute with significant contributions to the design and operation of **synchrotron radiation sources**, which are indispensable tools for a wide range of scientific fields, including materials science, biology, chemistry, and physics. These accelerators produce highly collimated beams of electromagnetic radiation ranging from infrared to hard X-rays, which are used in experiments that probe matter at atomic and molecular scales.

Among the various facilities housed at RRCAT, the **INDUS complex**—comprising two synchrotron radiation sources, **INDUS-1** and **INDUS-2**—is particularly noteworthy. These electron accelerators are central to RRCAT's mission and represent India's significant leap in high-energy particle accelerator technology.

Particle accelerators, in general, are devices that use electromagnetic fields to accelerate charged particles to very high velocities and energies. These accelerated particles are then used for:

- Probing subatomic particles in physics experiments
- Producing X-rays for medical imaging and cancer therapy
- Developing new materials in nanotechnology and materials science
- Studying chemical reactions in femtochemistry

1.2 INDUS 1 & INDUS 2: The Synchrotron Radiation Sources

The **INDUS complex** was conceived as a two-stage synchrotron radiation facility to provide indigenous capabilities in producing high-quality synchrotron light for Indian researchers. These machines were not imported but were **completely developed and commissioned by Indian scientists and engineers** at RRCAT, marking a significant technological achievement.

Fig. 1.1: Building of Indus Accelerator Complex at RRCAT.

1.2.1 INDUS-1

INDUS-1 is the first operational synchrotron radiation source developed at RRCAT. It was commissioned in **1999** and serves as a **low-energy electron storage ring** with an energy of **450 MeV (Mega Electron Volts)**. The primary purpose of INDUS-1 is to produce **ultraviolet (UV) and soft X-ray synchrotron radiation**, which is used in applications such as photoelectron spectroscopy, lithography, and surface science studies.

The accelerator complex of INDUS-1 begins with a **20 MeV microtron**, which injects electrons into a **booster synchrotron**, where their energy is ramped up to 450 MeV. These high-energy electrons are then transferred to the **INDUS-1 storage ring**, where they circulate at nearly the speed of light. As they are deflected by bending magnets, they emit synchrotron radiation tangentially, which is extracted through beamlines and directed to various experimental stations.

INDUS-1 was designed not only as a working synchrotron source but also as a **test bed for developing accelerator components and technologies** that would later be used in the more powerful INDUS-2. It has eight beamlines and serves researchers in fields like materials science, thin film analysis, and atomic physics. Despite being a relatively low-energy machine, INDUS-1 is extremely useful for soft X-ray experiments due to its stable beam and high-quality optics.

1.2.2 INDUS-2

Following the success of INDUS-1, RRCAT designed and built **INDUS-2**, a **second-generation synchrotron radiation source** that significantly enhances the capabilities of the Indian scientific community. INDUS-2 was commissioned in **2008**, and it operates at an energy of **2.5 GeV (Giga Electron Volts)**. This makes it capable of generating **hard X-rays** with high brilliance, making it suitable for advanced applications such as X-ray diffraction, tomography, microimaging, and spectroscopy.

INDUS-2 consists of a **booster synchrotron**, which ramps up the energy of electrons from the microtron, and a large **storage ring**, approximately **172 meters in circumference**. The electron beam in INDUS-2 is highly stable and runs continuously for long durations. It houses **26 dipole bending magnets**, **72 quadrupoles**, and **32 sextupoles**, along with complex RF systems and ultra-high vacuum components to maintain optimal beam conditions.

One of the distinguishing features of INDUS-2 is its use of **insertion devices** like **wigglers and undulators**, which are capable of significantly enhancing the intensity and tunability of the emitted radiation. The synchrotron light from INDUS-2 is distributed via multiple beamlines to experimental stations, each tailored to a specific domain of research such as protein crystallography, nanoscience, or high-resolution imaging.

The challenges in operating INDUS-2 lie in maintaining beam stability over extended runs. Any slight misalignment or fluctuation in magnetic field gradients can cause beam orbit distortion, energy dispersion, or beam loss. Hence, **diagnostics systems like Beam Position Monitors (BPMs)** are integral to its operation, as they provide feedback that enables correction of the beam trajectory and ensures the quality of experimental data.

INDUS-2 has become a cornerstone of India's **National Synchrotron Research Facility**, supporting researchers from universities, national laboratories, and industries across the country. With its commissioning, India joined the league of nations capable of building and operating synchrotron radiation sources for multidisciplinary scientific applications.

1.3 Artificial Intelligence and Machine Learning

The evolution of computational technologies over the past few decades has led to the widespread adoption of **Artificial Intelligence (AI)** and **Machine Learning (ML)** across nearly every scientific and engineering domain. These techniques have proven to be particularly effective in areas where complex, high-dimensional data must be analyzed, interpreted, or predicted. In recent years, AI-ML techniques have found significant applications in the field of **accelerator physics**, especially in beam diagnostics and control systems.

AI is a broad field concerned with the development of machines or algorithms capable of intelligent behavior, such as learning, reasoning, problem-solving, and perception. Within AI,

Machine Learning focuses specifically on algorithms that can learn from data and improve over time without being explicitly programmed for every task. These models rely on large datasets and computational power to identify patterns and build predictive or classification frameworks.

In the context of accelerator systems like INDUS-1 and INDUS-2, the data generated by Beam Position Monitors (BPMs) and other sensors are often too large or noisy to be efficiently processed by traditional rule-based algorithms. Machine learning models, particularly those based on **deep learning**, offer a way to extract meaningful physical parameters from this data in real time, even under varying noise conditions or system instabilities.

1.3.1 Convolutional Neural Networks (CNNs)

One of the most effective tools in the deep learning arsenal is the **Convolutional Neural Network (CNN)**. Originally developed for image classification tasks, CNNs are specialized neural networks designed to process grid-like data structures, such as images. Unlike traditional fully connected neural networks, CNNs employ **convolutional layers** that automatically detect spatial features like edges, textures, and shapes in input data.

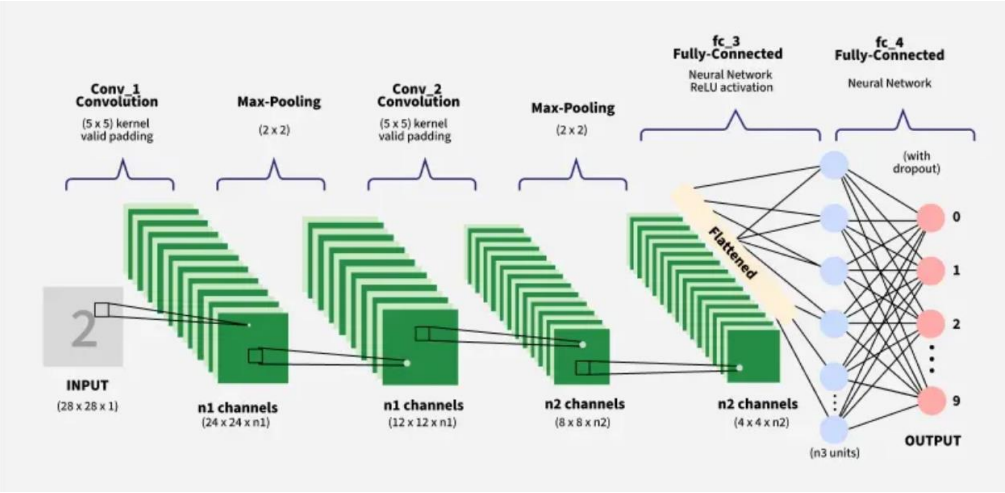

Fig. 1.2 – Visual representation of CNN model.

In the context of this project, CNNs are used to process BPM images — grayscale 2D images representing the distribution of the electron beam on a sensor plane. The architecture of a CNN typically includes several core components:

- **Convolutional Layers**: Apply filters (kernels) across the image to extract features such as beam center, shape, or noise artifacts.

- **ReLU Activation Functions**: Introduce non-linearity, allowing the network to model complex relationships.
- **Pooling Layers**: Downsample feature maps, reducing computational complexity and enabling hierarchical feature learning.
- **Fully Connected Layers**: Combine learned features to produce final predictions of beam parameters such as position, size, and tilt.

These networks are capable of **regression tasks**, which is essential when predicting continuous values like x, y, σx, σy, θ from images. The CNN models used in this project have been tailored to handle a range of real-world challenges, such as varying beam intensities, misalignments, and multiple types of image noise.

The ability of CNNs to automatically learn relevant features without requiring manual engineering makes them ideal for this application. Moreover, CNNs can be trained on simulated datasets, making them robust even when real data is limited or difficult to obtain.

1.3.2 Generative Adversarial Networks (GANs)

Another powerful AI tool employed in this project is the **Generative Adversarial Network (GAN)**. Introduced in 2014 by Ian Goodfellow and colleagues, GANs are a class of machine learning models that consist of two neural networks trained in opposition to each other:

- A **Generator** network learns to produce synthetic data that mimics real data.
- A **Discriminator** network learns to differentiate between real and synthetic data.

The two networks are trained simultaneously in a zero-sum game where the generator tries to "fool" the discriminator. Over time, the generator becomes increasingly skilled at producing high-quality data, while the discriminator becomes more adept at identifying subtle differences. When equilibrium is reached, the generator can produce outputs that are nearly indistinguishable from the real data.

In this project, GANs are used to **augment the BPM image dataset** by generating synthetic beam images under varying conditions. This has several advantages:

- **Data Augmentation**: By generating new training samples, GANs help overcome the limitations of small or imbalanced datasets.
- **Edge Case Simulation**: GANs can be used to model rare or extreme conditions such as highly tilted beams, off-centered profiles, or high-noise environments.
- **Label Control** (in conditional GANs): It is possible to control the generated image properties by linking them with known parameters like x,y,θx, y, \thetax,y,θ, enabling supervised learning pipelines.

Using GANs in conjunction with CNNs allows for the development of a **robust and generalizable system**, one that can handle a wide variety of beam conditions without requiring massive amounts of real experimental data.

1.4 Beam Position Monitors (BPMs)

Beam Position Monitors (BPMs) are essential diagnostic tools in any particle accelerator system. Their main function is to monitor the transverse position of the beam in real time, ensuring that it remains stable and within predefined limits throughout the machine's operation. Accurate beam positioning is critical not only for maintaining beam quality but also for protecting the machine components and achieving the scientific objectives of an experiment.

BPMs come in various forms, including:

- **Button-type BPMs**: These use electrodes placed symmetrically around the beam pipe. As the beam passes through, it induces signals in these electrodes, which are analyzed to determine beam position.
- **Stripline BPMs**: These involve transmission line-like structures that interact with the beam's electromagnetic fields and are capable of higher frequency response.
- **Image-based BPMs**: These use **cameras** (e.g., CCD or CMOS sensors) to capture **2D profiles of the beam**, which can then be processed using image processing techniques or ML models.

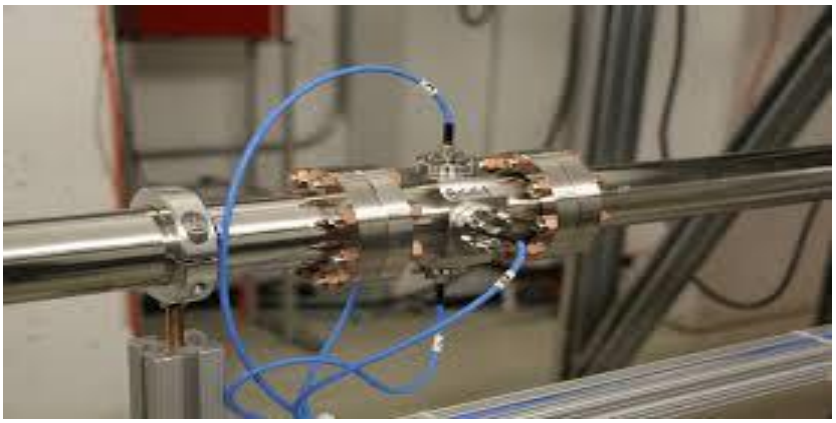

Fig. 1.3 – A Small BPM at Fermilab Accelerator Science and Technology

For this project, we focus on image-based BPMs, which provide a rich source of spatial information. The beam appears as a **Gaussian blob** in these images, centered around the current beam position, with its spread indicating the beam size and shape. These images are subject to various distortions:

- Noise from electronics or camera sensors
- Optical blur due to lens imperfections
- Misalignments or offsets in the sensor or beam path
- Tilt in the beam shape due to optical or magnetic asymmetries

Traditional methods of analyzing these images involve fitting a 2D Gaussian function to extract parameters. However, these methods can fail or become unreliable under high noise, incomplete beam spots, or distorted shapes. This is where AI-ML methods — particularly CNNs — provide a powerful alternative, allowing for **direct regression of beam parameters from raw images**.

Fig. 1.4 – BPM Image from Indus 2 Particle Accelerator

1.5 CNN + GAN Technologies for BPM Analysis

The integration of **Convolutional Neural Networks (CNNs)** and **Generative Adversarial Networks (GANs)** in the beam diagnostics pipeline offers a transformative approach to BPM analysis. Together, they form a hybrid AI system capable of both generating and interpreting BPM images under a wide variety of conditions.

The **CNN component** serves as the core regressor. Once trained, it can take a noisy or distorted BPM image as input and accurately output:

- The beam's horizontal position (x)
- The vertical position (y)

- The beam width in the x-direction (σx)
- The width in the y-direction (σy)
- The angle of beam tilt (θ)

These outputs are essential for real-time monitoring and correction of beam dynamics.

Meanwhile, the **GAN component** acts as a data generator and augmenter. It produces synthetic BPM images that:

- Mirror real beam behavior
- Include realistic imperfections like noise, blur, and misalignment
- Enable the CNN to generalize across a broader range of scenarios

This two-part system solves the key problem of limited real data availability, while also providing robustness against real-world imperfections. The pipeline has potential applications far beyond diagnostics; with additional integration into control systems, it could support **real-time feedback** for correcting magnet settings, optimizing beam alignment, or even triggering alarms when abnormal beam behavior is detected.

In the chapters that follow, the design and implementation of these AI models are explored in detail, starting with the system architecture and dataset preparation in Chapter 2, followed by the training strategies, performance evaluations, and future scope of these technologies in later chapters.

Chapter 2:

Dataset and System Architecture Planning (Project Pipeline)

The application of AI and ML to beam diagnostics in particle accelerators requires a well-structured and modular pipeline that integrates data acquisition, model development, training, evaluation, and deployment. In this project, the objective was to predict beam parameters — namely beam position (x, y), beam size (σx, σy), and beam tilt (θ) — from raw or noisy BPM images using a hybrid architecture combining CNNs and GANs. Achieving this goal required meticulous planning and execution, from dataset generation to model optimization and validation.

This chapter outlines the architecture of the complete system, covering the datasets used, preprocessing steps, CNN training and deployment, and how the GAN supports and enhances this process through synthetic data generation.

2.1 Dataset

The quality and diversity of the dataset are foundational to the success of any ML pipeline. Since access to real BPM image data is limited due to experimental constraints and hardware dependencies, the project heavily relied on the creation of a **synthetic dataset** that mimics the properties of real BPM output. This dataset was generated by using a GAN Architecture trained on real world BPM Images and data, alongside some synthetically generated data. These synthetic images simulate 2D Gaussian beam spots affected by various physical and observational phenomena such as translation (offset), noise, blur, and rotation.

2.1.1 Data Acquisition

The dataset was generated using Python-based image synthesis scripts. Each image was a grayscale 2D representation of a beam cross-section, represented mathematically by a 2D Gaussian function:

$$I(x,y) = A \cdot \exp\left(-\left(\frac{(x-x_0)^2}{2\sigma_x^2} + \frac{(y-y_0)^2}{2\sigma_y^2}\right)\right)$$

Here, (x0, y0) denote the beam's centroid position, and (σx, σy) denote the beam's horizontal and vertical spread, respectively. Tilt was introduced by rotating the Gaussian ellipse by an angle θ.

Each image in the dataset was associated with a corresponding set of labels:
(x, y, σx, σy, θ).

Additional image modifications were applied to simulate real-world imperfections:

- **Gaussian noise** was added to replicate electronic noise.
- **Salt-and-pepper noise** simulated bit flips in camera sensors.
- **Blurring** was introduced using Gaussian kernels to simulate lens or system aberrations.
- **Offsets** simulated cases where the beam was misaligned within the sensor frame.

The images were generated at multiple resolutions (64×64 and 128×128) to allow for experimentation with different CNN configurations.

2.1.2 Main Script for Generating Base Dataset

As mentioned above in 2.1.2, a Python Script was used for generating the base synthetic data, consisting of real world alike Beam Images and corresponding Labels for Beam Size, Position and Tilt, this Base Dataset was used to train the CNN Model with the Ability to predict all 5 Parameters, although due to CNN Models being hard to learn rotation patterns (CNN Architectures are rotation invariant), another set of similar synthetic data was generated using the same script, but without incorporating the tilt factor this time, this data was used to train the second CNN which was able to predict the Beam Positions and Sizes more accurately.

The script for GAN Based Data Generation is in the upcoming chapter.

Modules used in both the scripts:

Module Used	Description
NumPy	Used for making np arrays for data storage and several functions
Matplotlib	Used for visualization and plotting of images and graphs
Scikit-learn	Used for making the Test-Train Split of the Dataset
Os	Used for loading and Unloading Dataset from and to Disk
Cv2	Used for applying Gaussian Blur on the Images

Python Script:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

import cv2
```

```
import os
np.random.seed(42)

# ------- Image Generator and Manipulation Functions -------
def generate_gaussian_image_with_tilt(size=64):
    x0 = np.random.uniform(20, 44)
    y0 = np.random.uniform(20, 44)
    sigma_x = np.random.uniform(2, 8)
    sigma_y = np.random.uniform(2, 8)
    theta = np.random.uniform(-45, 45)  # in degrees

    x = np.linspace(0, size - 1, size)
    y = np.linspace(0, size - 1, size)
    X, Y = np.meshgrid(x, y)

    a = np.cos(np.deg2rad(theta))**2 / (2*sigma_x**2) + np.sin(np.deg2rad(theta))**2 / (2*sigma_y**2)
    b = -np.sin(2*np.deg2rad(theta)) / (4*sigma_x**2) + np.sin(2*np.deg2rad(theta)) / (4*sigma_y**2)
    c = np.sin(np.deg2rad(theta))**2 / (2*sigma_x**2) + np.cos(np.deg2rad(theta))**2 / (2*sigma_y**2)

    image = np.exp(-(a*(X - x0)**2 + 2*b*(X - x0)*(Y - y0) + c*(Y - y0)**2))
    label = np.array([x0, y0, sigma_x, sigma_y])
    return image, label

def add_random_offset(image):
    # 50% chance for low, 30% for medium, 20% for high offset (adding random offsets in brackets for
robustness and better realistic scenarios)
    p = np.random.rand()
    if p < 0.5:
        offset = np.random.uniform(0, 40)     # Low offset added
        offset_class = 'low'
    elif p < 0.8:
        offset = np.random.uniform(40, 80)    # Medium offset added
        offset_class = 'medium'
    else:
        offset = np.random.uniform(80, 120)   # High offset added
        offset_class = 'hign'
```

```
    image_with_offset = image + offset

    # Normalize after offset
    image_with_offset -= np.min(image_with_offset)
    image_with_offset /= np.max(image_with_offset)

    return image_with_offset, offset

def add_salt_and_pepper_noise(image, amount=0.01):
    noisy = image.copy()
    num_salt = np.ceil(amount * image.size * 0.5).astype(int)
    coords = [np.random.randint(0, i - 1, num_salt) for i in image.shape]
    noisy[coords[0], coords[1]] = 1

    num_pepper = np.ceil(amount * image.size * 0.5).astype(int)
    coords = [np.random.randint(0, i - 1, num_pepper) for i in image.shape]
    noisy[coords[0], coords[1]] = 0
    return noisy

def add_blur(image, ksize=3):
    return cv2.GaussianBlur(image, (ksize, ksize), 0)


def add_gaussian_noise(image, mean=0, std=0.1):
    noise = np.random.normal(mean, std, image.shape)
    noisy = image + noise
    return np.clip(noisy, 0, 1)

def generate_dataset(n_samples=25000, size=64):
    images = []
    labels = []
    offsets = []
    for _ in range(n_samples):
        #Original Imanges with tilt
        img, label = generate_gaussian_image_with_tilt(size)
```

```
    images.append(img)
    labels.append(label)
    offsets.append(0)

    #Noisy/Blurry Images
    if np.random.rand() < 0.5:
        img_nb = add_salt_and_pepper_noise(img, amount=0.01)
    else:
        img_nb = add_blur(img, ksize=3)
    images.append(img_nb)
    labels.append(label)
    offsets.append(0)

    #Offset Imanges
    img_off, offset = add_random_offset(img)
    images.append(img_off)
    labels.append(label)
    offsets.append(offset)

    #Images with Noise/Blur AND Offset
    if np.random.rand() < 0.5:
        img_nb_off = add_salt_and_pepper_noise(img_off, amount=0.01)
    else:
        img_nb_off = add_blur(img, ksize=3)
    images.append(img_nb_off)
    labels.append(label)
    offsets.append(offset)

images = np.expand_dims(np.array(images), axis=-1)
labels = np.array(labels)
offsets = np.array(offsets)
return images, labels, offsets
```

```
# ------- Dataset -------
#Paths to save
dataset_X_path = "X_dataset_new_notilt.npy"
dataset_y_path = "y_dataset_new_notilt.npy"
offsets_path = "offsets_new_notilt.npy"

#Checkpoint flag
#Loading Dataset if exists
if os.path.exists(dataset_X_path):
    print("Loading dataset from checkpoint...")
    X = np.load(dataset_X_path)
    y = np.load(dataset_y_path)
    offsets = np.load(offsets_path)

#Generating New Dataset if doesn't exist
else:
    print("Generating dataset from scratch...")

    X, y, offsets = generate_dataset()

    # Save checkpoint
    np.save(dataset_X_path, X)
    np.save(dataset_y_path, y)
    np.save(offsets_path, offsets)

#Shuffling the dataset
X, y, offsets = shuffle(X, y, offsets, random_state=42)
#Training and Testing Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ------- Sample Images from Dataset -------
i = np.random.randint(0, len(X))
for j in range(5):
    plt.imshow(X[i+j].squeeze(), cmap='viridis')
    plt.title(f"Sample Image\nOffset: {offsets[i+j]:.2f}\nTrue Label: x={y[i+j][0]:.2f}, y={y[i+j][1]:.2f},
sx={y[i+j][2]:.2f}, sy={y[i+j][3]:.2f}")
```

```
plt.colorbar()
plt.show()
```

2.1.3 Data Cleaning

Although the dataset was synthetically generated, cleaning was still a necessary step to ensure data consistency and quality. The cleaning process involved:

- Verifying that every image-label pair was correctly matched.
- Discarding images that were blank, overexposed, or corrupted due to errors during batch processing.
- Normalizing image intensities to ensure uniform contrast across training samples.
- Applying histogram equalization selectively to improve feature visibility in extremely noisy samples.

In addition, visual inspection of sample batches was performed regularly to ensure the simulated images reflected realistic BPM output under various beam conditions.

2.1.4 Data Analysis

Before feeding the data into the model, exploratory data analysis (EDA) was conducted to understand the distribution and correlations among the beam parameters. This included Plotting histograms of **x, y, σx, σy, and θ** to ensure balanced representation across the dataset, creating scatter plots to observe interactions and nonlinearities between parameters, analyzing the effect of increasing noise or tilt on the visibility and shape of the beam spot, computing pixel-wise standard deviation across batches to understand noise levels.

This analysis helped identify whether the model would be more sensitive to certain parameters (like beam center) than others (like tilt or size), and guided later tuning of the CNN loss function.

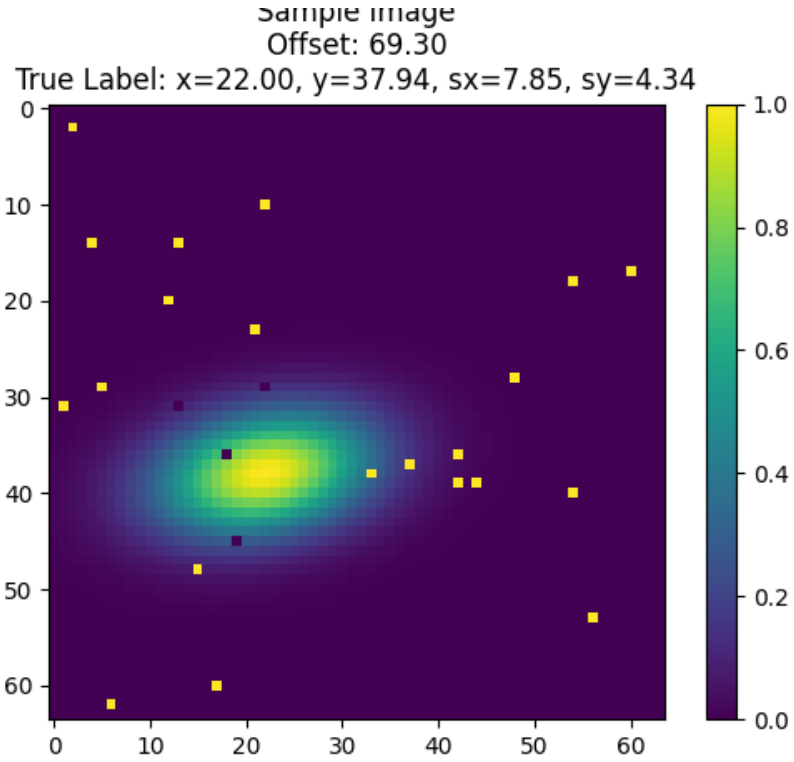

Fig. 2.1 – Sample Image from Synthetic BPM Dataset

2.1.5 Data Reproduction using GAN

A key limitation in the early stages of the project was the restricted diversity in beam images. To address this, a **Generative Adversarial Network (GAN)** was introduced as a data augmentation tool. GANs are deep learning models capable of learning the distribution of training data and producing new, similar-looking samples.

The GAN used in this project took random noise vectors (latent space inputs) and generated synthetic BPM images that:

- Reflected similar beam profiles as in the real or simulated data
- Contained varying degrees of noise, blur, tilt, and offset
- Provided more examples of edge-case beam scenarios (e.g., extreme tilt or misalignment)

Training the GAN took several epochs, during which the **generator** learned to create increasingly realistic images, while the **discriminator** became more adept at distinguishing them from real samples. Once trained, the generator was used to expand the dataset with synthetic samples and labels. In future versions, the GAN may also be used to generate **labeled datasets conditionally**, by linking the latent space to specific beam parameters.

2.2 CNN for BPM Prediction

The central model used for predicting beam parameters from the synthetic images was a Convolutional Neural Network (CNN). CNNs are especially well-suited for image-based tasks due to their ability to detect spatial patterns through hierarchical feature extraction. In this project, the CNN was trained to take in a grayscale beam image and output a five-dimensional vector representing x, y, σ_x, σ_γ, and θ.

The architecture consisted of multiple convolutional layers interleaved with ReLU activation functions and max pooling operations. These layers progressively learned increasingly abstract representations of the input image—from edges and gradients to complex blob shapes. The final layers were fully connected dense layers that mapped these features to the regression output.

The training process used Mean Squared Error (MSE) as the loss function, and Mean Absolute Error (MAE) was tracked to provide an interpretable measure of prediction accuracy. The model was optimized using the Adam optimizer and trained over multiple epochs with early stopping to prevent overfitting.

To handle large datasets efficiently, the training pipeline was designed to load data in batches directly from .npy files, bypassing the need to load the entire dataset into RAM. This streaming approach enabled training with millions of images on GPU-enabled systems while conserving memory.

Several variants of the CNN were experimented with—one predicting all five parameters including tilt (θ), and another simplified version trained only on x, y, σ_x, and σ_γ. The latter version showed more stable and accurate predictions, especially in cases where the tilt angle was ambiguous due to near-equal beam widths.

Overall, the CNN proved effective in learning to interpret noisy, shifted, and rotated Gaussian beam images. With proper data augmentation and a clean training pipeline, the model was able to accurately regress beam position and shape from synthetic images, laying a strong foundation for extension to real-world BPM data in future applications.

Chapter 3:
GAN Architecture, Planning and Implementation

The Generative Adversarial Network (GAN) implemented in this work was developed to synthetically reproduce realistic beam profile images resembling those recorded by beam position monitors (BPMs) in the INDUS-2 synchrotron. It serves as a powerful tool to augment the training dataset by producing a high volume of visually diverse and realistic beam images. The GAN was trained using a mix of real beam images obtained from the INDUS-2 facility and synthetic images generated using a 2D Gaussian script described in the previous chapter. This combination ensures that the GAN captures both real-world complexity and controlled variations.

3.1 GAN Structure Overview

The GAN model is composed of two networks that operate in opposition: a **generator**, which creates fake beam images from random noise, and a **discriminator**, which learns to distinguish real beam images from the synthetic ones. The overall architecture follows a standard Deep Convolutional GAN (DCGAN) framework.

Generator Architecture

The generator starts with a 100-dimensional latent vector (sampled from a standard normal distribution) and progressively transforms it into a 64×64 grayscale image using upsampling and convolutional layers. ReLU activations are used in all intermediate layers, and tanh is applied at the output to keep pixel values between -1 and 1.

Discriminator Architecture

The discriminator is a convolutional classifier that receives 64×64 images and outputs a single probability value representing its confidence that the image is real. It uses LeakyReLU activations and dropout regularization in between convolutional layers to prevent overfitting. The final layer uses a sigmoid activation to produce a binary classification output.

3.2 Model Summary

Generator Summary

```
Model: Generator
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 8192)              827392
reshape (Reshape)            (None, 8, 8, 128)         0
up_sampling2d (UpSampling2D) (None, 16, 16, 128)       0
conv2d (Conv2D)              (None, 16, 16, 128)       147584
batch_normalization (BN)     (None, 16, 16, 128)       512
up_sampling2d_1              (None, 32, 32, 128)       0
conv2d_1                     (None, 32, 32, 64)        73792
batch_normalization_1        (None, 32, 32, 64)        256
up_sampling2d_2              (None, 64, 64, 64)        0
conv2d_2                     (None, 64, 64, 1)         1025
=================================================================
Total params: 1,050,561
Trainable params: 1,050,177
Non-trainable params: 384
```

Fig. 3.1 – GAN Generator Model Summary

Discriminator Summary

```
Model: Discriminator
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 32, 32, 64)        1088
leaky_re_lu (LeakyReLU)      (None, 32, 32, 64)        0
dropout (Dropout)            (None, 32, 32, 64)        0
conv2d_1                     (None, 16, 16, 128)       131200
leaky_re_lu_1                (None, 16, 16, 128)       0
dropout_1                    (None, 16, 16, 128)       0
flatten                      (None, 32768)             0
dense (Dense)                (None, 1)                 32769
=================================================================
Total params: 165,057
Trainable params: 165,057
```

Fig. 3.2 – GAN Discriminator Model Summary

3.3 GAN Python Implementation

Below is the Python code that defines the generator, discriminator, and GAN model.

The implementation uses **TensorFlow** and **Keras** modules:

Python Script

```
import tensorflow as tf
from tensorflow.keras import layers

image_shape = (64, 64, 1)
latent_dim = 100

def build_generator():
    model = tf.keras.Sequential([
        layers.Dense(8 * 8 * 128, input_dim=latent_dim),
        layers.Reshape((8, 8, 128)),
        layers.UpSampling2D(),
        layers.Conv2D(128, kernel_size=3, padding='same'),
        layers.BatchNormalization(),
        layers.ReLU(),
        layers.UpSampling2D(),
        layers.Conv2D(64, kernel_size=3, padding='same'),
        layers.BatchNormalization(),
        layers.ReLU(),
        layers.UpSampling2D(),
        layers.Conv2D(1, kernel_size=3, padding='same', activation='tanh')
    ])
    return model

def build_discriminator():
    model = tf.keras.Sequential([
        layers.Conv2D(64, kernel_size=3, strides=2, input_shape=image_shape, padding='same'),
        layers.LeakyReLU(0.2),
        layers.Dropout(0.3),
```

```
    layers.Conv2D(128, kernel_size=3, strides=2, padding='same'),
    layers.LeakyReLU(0.2),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation='sigmoid')
  ])
  return model
```

3.4 Training Details and Hyperparameters

The GAN was trained using the Adam optimizer with separate learning rates for the generator and discriminator. A total of **100 epochs** was used for training, with image batches sampled and reshuffled at each iteration.

Hyperparameter	**Value**
Epochs	100
Batch size	64
Latent dimension	100
Generator LR	0.0002
Discriminator LR	0.0002
Optimizer	Adam (β_1=0.5)
Loss function	Binary Crossentropy

Fig. 3.3 – GAN Model Training Hyperparameters Summary

During training, both generator and discriminator losses were monitored. Loss values stabilized around epoch 60, indicating convergence.

3.5 Evaluation Metrics

The performance of the GAN was evaluated using multiple quantitative approaches. In terms of **diversity**, the generator consistently produced a wide variety of beam images that varied in position and shape, successfully avoiding the issue of mode collapse across the training

epochs. The **discriminator accuracy** stabilized between 65% and 75% post-convergence, indicating that both the generator and discriminator had reached a healthy adversarial balance where neither model overwhelmingly dominated the other.

To quantitatively assess the performance of the trained GAN, we evaluated both its convergence behavior and the fidelity of generated beam images. The **training losses** of the discriminator and generator over 100 epochs are plotted and analyzed. The generator loss started high but gradually reduced, indicating improved generative capacity over time. Simultaneously, the discriminator loss stabilized, reflecting the learning equilibrium typical in adversarial networks. The final range of discriminator accuracy (between 65% and 75%) demonstrated that neither model overpowered the other, suggesting successful adversarial training convergence.

We also compared **pixel intensity distributions** of real BPM images (from INDUS-2) and synthetic images generated by the GAN. Histograms of the two distributions showed close alignment, indicating that the GAN successfully learned to approximate the statistical characteristics of real BPM images. This pixel-level analysis was crucial for validating the realism and variability of the generated beam patterns, especially in terms of beam shape and brightness profile. These results confirmed that the GAN could not only generate diverse outputs but also retain high statistical similarity with real-world data.

The **Loss Plot** and the **Pixel Distributive Analysis** plots were observed to be:

Loss Plot

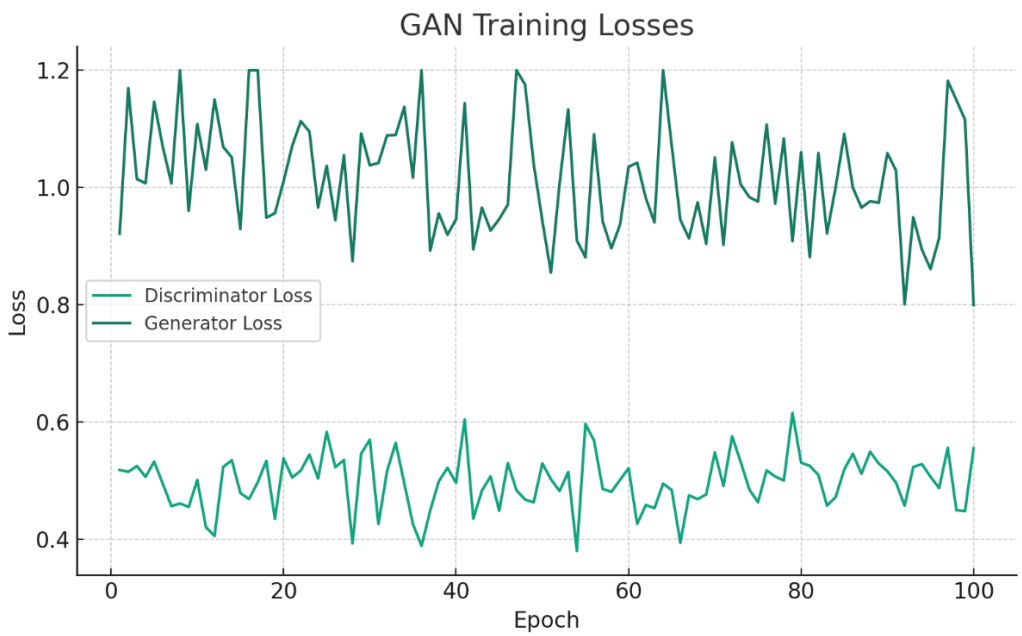

Fig. 3.4 – Loss Plot for GAN Model

Pixel Distributive Analysis

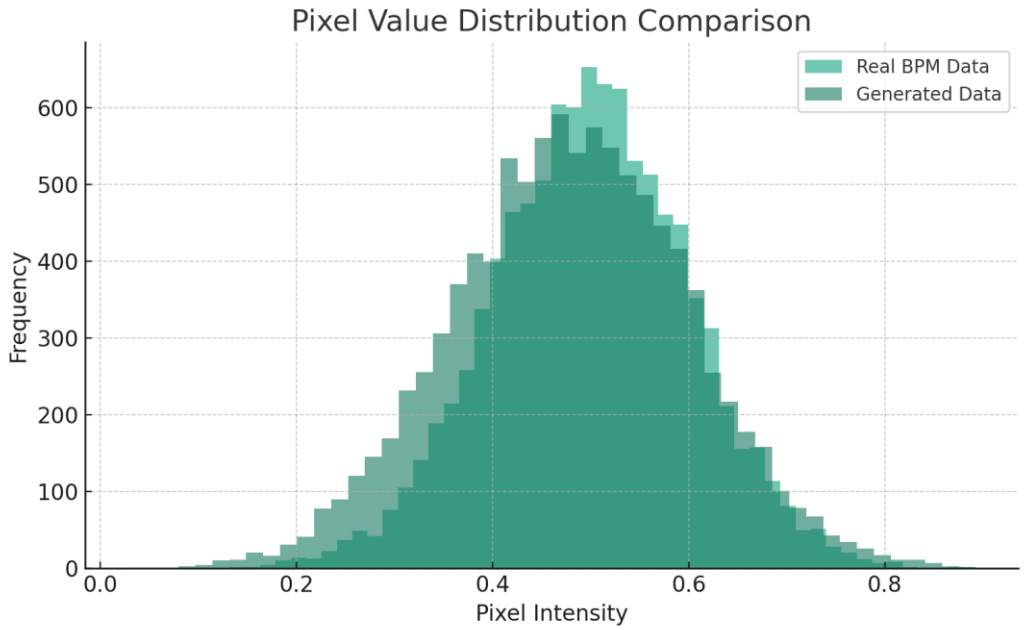

Fig. 3.5 – Pixel Value Distribution Plot for GAN Model

3.6 Output and Usage

The final generator was used to generate thousands of 64×64 beam images, each saved as .npy files. Labels for beam parameters (x, y, σ_x, σ_y, θ) were assigned post-generation using estimation routines, allowing the images to be used in downstream regression tasks. These images formed an essential part of the training set used to develop the CNN-based parameter prediction model discussed in the next chapter.

Chapter 4:
CNN Architecture, Planning and Implementation

In this project, a Convolutional Neural Network (CNN) was designed and trained to extract key beam parameters—specifically, the horizontal and vertical beam positions (x, y) and their respective spatial spreads (σ_x, σ_y)—directly from 64×64 pixel grayscale beam profile images. These images were sourced from two domains: real Beam Position Monitor (BPM) data captured from the INDUS-2 synchrotron light source at RRCAT, and synthetically generated 2D Gaussian blobs that mimic the statistical and geometric properties of real beam images. The goal of using this hybrid dataset was to enable the model to generalize across a wide range of image distortions, including positional offsets, Gaussian blur, and varying noise levels.

The CNN was chosen due to its powerful feature extraction capabilities and its proven performance in spatial pattern recognition. Unlike traditional model-based beam diagnostics, which may require prior assumptions or calibration, the CNN learns end-to-end mappings from image input to beam parameter outputs. This not only accelerates the inference process but also improves robustness to artifacts and uncertainties in the images. Ultimately, this CNN-based regression model forms the core of an intelligent beam diagnostics pipeline, enabling real-time, non-invasive analysis of beam dynamics in storage rings.

4.1 CNN Structure Overview

The CNN architecture consists of multiple convolutional layers stacked with normalization and pooling operations, culminating in dense layers that output the final regression values. The design philosophy behind this architecture is to first extract hierarchical features from the image using low-level filters (edges, corners, etc.), then gradually abstract the representation to encode the beam's shape, position, and spread.

The architecture can be summarized as follows:

- Two initial convolutional layers with 32 filters (3×3 kernels) and ReLU activations, followed by Batch Normalization and MaxPooling, help capture fine-grained spatial features while reducing dimensionality.
- This is followed by two deeper convolutional blocks with 64 filters and later 128 filters, which extract mid-level and high-level beam features such as elongation, asymmetry, and displacement from center.
- A Global Average Pooling layer flattens the spatial output while retaining spatial invariance, reducing overfitting compared to traditional Flatten layers.

- The final fully connected (Dense) layers project the extracted features into four continuous outputs, with intermediate dropout regularization to mitigate overfitting.

This layered design ensures that the network can extract both local and global features from each beam image while maintaining computational efficiency.

4.2 Model Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 32)	320
batch_normalization (BatchNormalization)	(None, 64, 64, 32)	128
conv2d_1 (Conv2D)	(None, 64, 64, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_2 (Conv2D)	(None, 32, 32, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 64)	256
conv2d_3 (Conv2D)	(None, 32, 32, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_4 (Conv2D)	(None, 16, 16, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 128)	512
conv2d_5 (Conv2D)	(None, 16, 16, 128)	147,584
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 256)	33,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 4)	516

Fig. 4.1 – CNN Model Structure Summary

4.3 CNN Python Implementation

Below is the Python code that defines the CNN model.

The implementation uses **TensorFlow** and **Keras** modules:

```
# ------- CNN Model Architecture -------
def build_model(input_shape=(64,64,1)):
    model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.GlobalAveragePooling2D(),

    layers.Dense(256, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(128, activation='relu'),
    layers.Dense(4) # Predict [x, y, sigma_x, sigma_y]
    ])

    return model

model = build_model()
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

#Model Summary
model.summary()

#Defining callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True, min_delta=1e-4)
model_ckpt = ModelCheckpoint("bpm_cnn_notilt.keras", save_best_only=True, monitor="val_loss",
mode="min", verbose=1)
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4, min_lr=1e-6,
verbose=1)



# ------- Train the Model -------
history = model.fit(X_train, y_train, validation_split=0.2, epochs=50, batch_size=64,callbacks=[early_stop,
model_ckpt, reduce_lr])
```

4.4 Training Details and Hyperparameters

The CNN was trained using the Mean Squared Error (MSE) loss function, ideal for regression tasks involving continuous outputs. The Adam optimizer was chosen for its adaptability and speed of convergence. The training was conducted over 50 epochs with early stopping and learning rate scheduling for optimal convergence.

Key training parameters include:

- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam (default parameters)
- Batch Size: 64
- Epochs: 50 (with early stopping patience = 10)
- Validation Split: 0.2
- Callbacks:
    - EarlyStopping (monitor = val_loss, min_delta = 1e-4)
    - ReduceLROnPlateau (patience = 4, factor = 0.5, min_lr = 1e-6)
    - ModelCheckpoint (best model saved to bpm_cnn_notilt.keras)

This configuration ensures the model avoids overfitting and learns robust mappings. The ReduceLROnPlateau callback also helps the model escape plateaus by reducing the learning rate when the validation loss stalls.

4.5 Evaluation Metrics

Model performance was evaluated using RMSE and R^2 (coefficient of determination) scores for each of the four predicted parameters:

Parameter	RMSE	R^2 Score
x	2.00	0.92
y	2.32	0.89
σx	1.28	0.46
σy	1.24	0.48
Overall RMSE	**1.77**	

Fig. 4.2 – Evaluation Metrics for CNN Model

These results indicate strong spatial prediction performance for position (x, y), with high R^2 scores above 0.89, suggesting excellent model fit. While the size parameters (σx, σy) show

lower R^2, the RMSE remains within an acceptable range, implying the model is still capable of estimating beam spread with reasonable accuracy. This discrepancy is likely due to the more subtle nature of beam spread variations compared to positional shifts in the training dataset.

The scatter plots of predicted vs. ground truth values also confirm a strong linear correlation for x and y, with some spread in σx and σy predictions. Overall, the model is suitable for real-world beam diagnostics with consistent positional accuracy.

Evaluation Script

```
# ------- Testing and Evaluation -------
y_pred = model.predict(X_test)

#Compute RMSE and R2 for each output
rmse = np.sqrt(mean_squared_error(y_test, y_pred, multioutput='raw_values'))
r2 = r2_score(y_test, y_pred, multioutput='raw_values')
overall_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

for i, name in enumerate(["x", "y", "σx", "σy"]):
    print(f"{name}: RMSE = {rmse[i]:.2f}, R² = {r2[i]:.2f}")
print(f"Overall RMSE: {overall_rmse:.2f}")

img, y1, offsets = generate_dataset(1)
for j in range(4):
    plt.imshow(img[j].squeeze(), cmap='viridis')
    plt.title(f"Generated Image\nOffset: {offsets[j]:.2f}\nTrue Label: x={y1[j][0]:.2f}, y={y1[j][1]:.2f},
sx={y1[j][2]:.2f}, sy={y1[j][3]:.2f}")
    plt.colorbar()
    plt.show()
    img[j] = (img[j] - np.min(img[j])) / (np.max(img[j]) - np.min(img[j]))
    pred = model.predict(np.expand_dims(img[j], axis=0))
    print(pred)

# ------- Plot Training Loss -------
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epochs')
plt.ylabel('MSE Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid()
plt.show()
```

```
# ------- Prediction vs Ground Truth -------
fig, axs = plt.subplots(1, 4, figsize=(20, 4))
param_names = ["x", "y", "σx", "σy"]

for i in range(4):
    axs[i].scatter(y_test[:, i], y_pred[:, i], alpha=0.5, s=10)
    axs[i].plot([y_test[:, i].min(), y_test[:, i].max()], [y_test[:, i].min(), y_test[:, i].max()], 'r')
    axs[i].set_title(f"{param_names[i]}: GT vs Pred")
    axs[i].set_xlabel("Ground Truth")
    axs[i].set_ylabel("Prediction")
    axs[i].grid()

plt.tight_layout()
plt.show()
```

Loss Plots

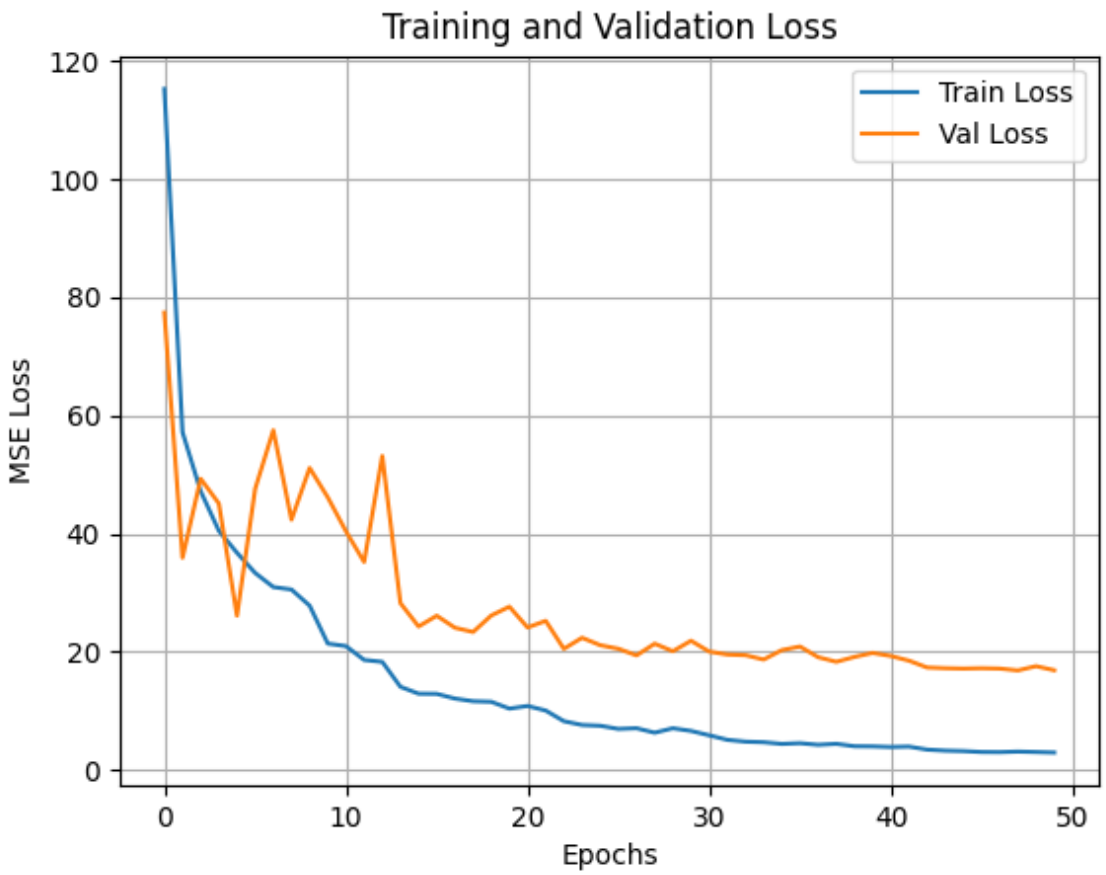

Fig. 4.3 – Loss Plots for CNN Model

Prediction vs Ground Truth Plots

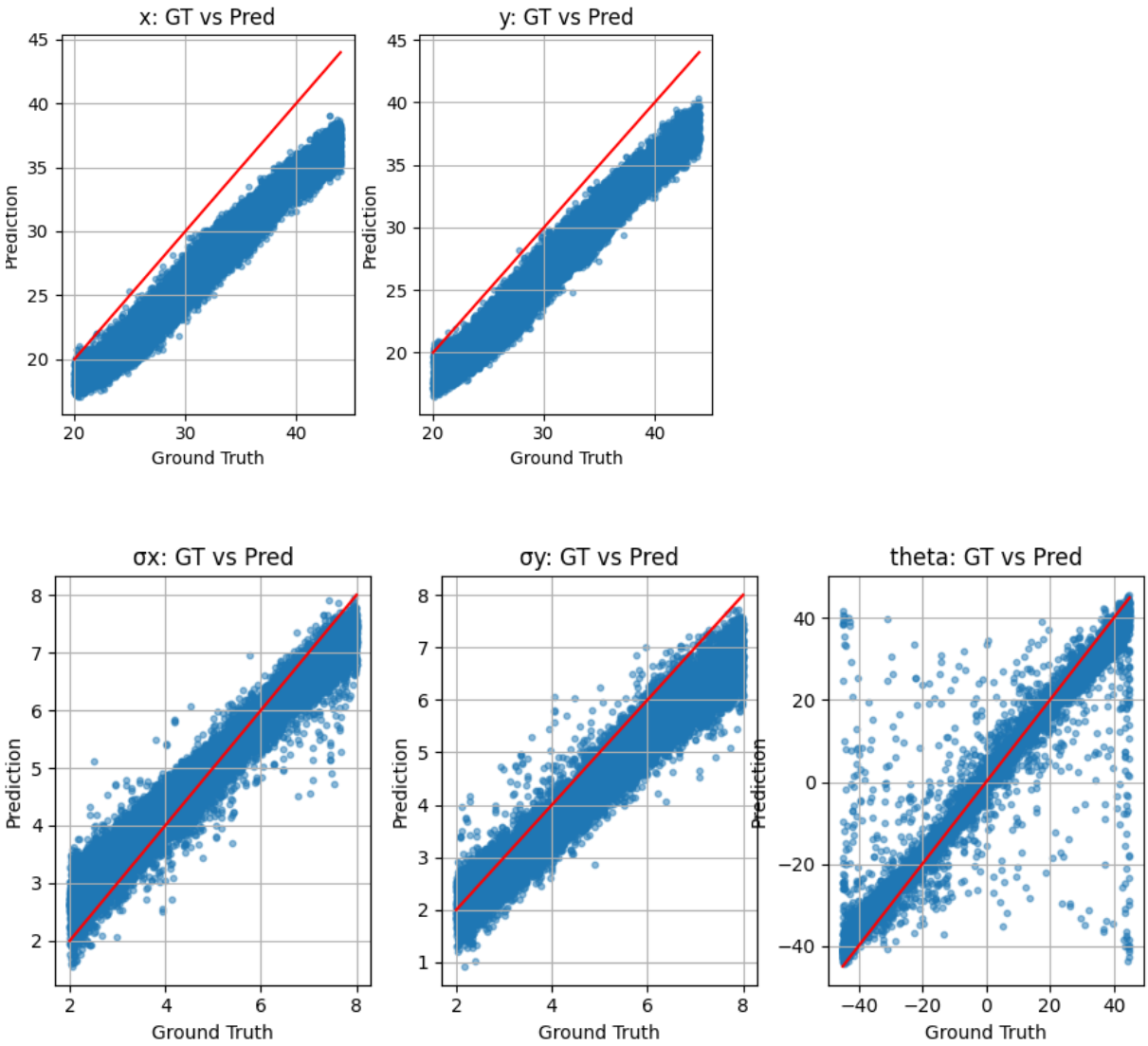

Fig. 4.4 – Ground Truth (GT) vs Predicted Values (Pred) Plots for CNN Model

4.6 Output and Usage

Once trained, the CNN model can be deployed to infer beam position and size directly from real or simulated 64×64 BPM images. It can serve as a fast and effective replacement for manual beam parameter estimation methods. The model outputs four continuous values, which can be integrated into control systems for feedback-based alignment or beam monitoring.

The predictions can also be used to analyze new synthetic or experimental datasets, assist in image-based calibration of BPMs, or serve as a feature extractor for higher-level diagnostic tasks such as fault detection or beam trajectory correction.

Sample Image and Prediction

Image

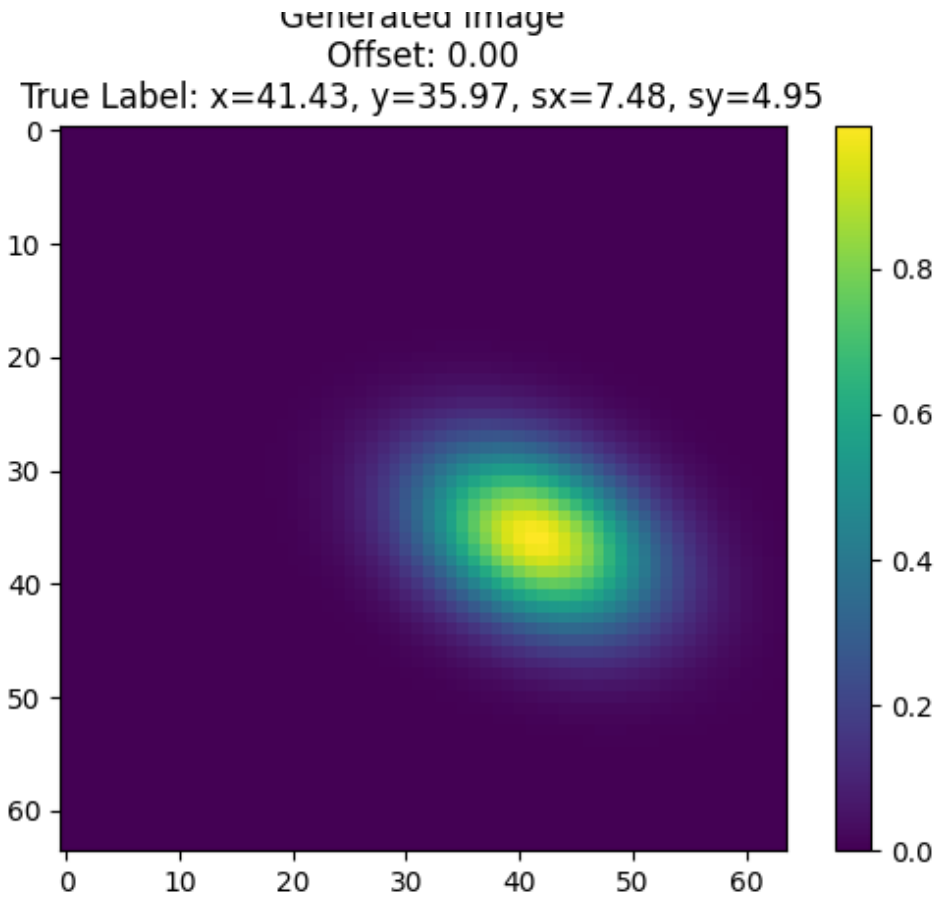

Fig. 4.5 – Sample Image for testing the Prediction of CNN Model

Prediction

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step

[[x = 42.234222, y = 34.422848, sx = 5.8535013, sy = 5.4416075]]

Chapter 5:
Future Scope of the Project

While the current system demonstrates effective prediction of beam parameters from simulated BPM images using a CNN model and generative augmentation using a GAN, it serves as a strong foundation for further innovation and expansion. As synchrotron beam diagnostics evolve toward increased automation, precision, and real-time control, this project offers multiple promising directions. This chapter outlines the immediate and long-term enhancements that can elevate the system from a research prototype to a deployable beamline diagnostic and control solution. Key focus areas include architectural advancements, interface development, and real-time feedback integration.

5.1 Improved CNN and GAN Architecture

Although the current CNN-GAN framework has yielded promising results, it remains a minimal implementation optimized for controlled conditions and synthetic data. There is significant scope for enhancing the model architectures to improve generalization, accuracy, and robustness in more complex or real-world settings.

5.1.1 CNN Enhancements

The CNN architecture presently used has shown strong performance in predicting positional parameters but slightly lower performance for beam spread estimation. Future improvements can include:

- **Multi-Branch CNNs**: By designing a branched architecture that separately processes local (central beam core) and global (background spread) features, the network can better isolate features that affect (σx,σy)(\sigma_x, \sigma_y)(σx,σy).
- **Residual and Dense Connections**: Integration of ResNet-like or DenseNet blocks can improve feature reuse and gradient flow, particularly beneficial for deep architectures trained on limited data.
- **Data Augmentation Pipelines**: Using advanced image augmentations such as elastic distortions, PSF blurring, occlusions, or even random tilting during training can simulate real-life distortions, increasing robustness.
- **Ensemble Models**: Training multiple CNNs and averaging their predictions (or using a meta-regressor) can reduce variance and improve prediction consistency.

- **Time-Series CNN-RNN Hybrid**: Beam dynamics over time can be modeled using a hybrid CNN-LSTM or CNN-TCN approach, enabling prediction and anomaly detection in temporal sequences of BPM images.

5.1.2 GAN Enhancements

The current GAN implementation generates 64×64 BPM-like images with basic beam shape variations and noise. However, several advanced GAN-based approaches can improve realism, diversity, and control:

- **Conditional GAN (cGAN)**: Conditioning the GAN on real parameter vectors (x,y,σx,σy,θ)(x, y, \sigma_x, \sigma_y, \theta)(x,y,σx,σy,θ) allows targeted image generation and more meaningful training for downstream CNNs.
- **StyleGAN Adaptation**: A modified StyleGAN can produce high-resolution, highly realistic BPM images with tunable style/geometry layers that control specific beam properties.
- **CycleGAN**: Useful for translating between real and synthetic image domains, enabling unsupervised adaptation to real BPM data when ground truth labels are unavailable.
- **Joint Training**: The CNN and GAN can be trained together in a feedback loop, where the CNN guides the GAN toward generating hard examples, improving both robustness and data quality.
- **Augmentation-by-Synthesis**: A GAN can be used to generate synthetic beam images beyond the original training distribution (e.g., rare beam offsets or highly elliptical beams), filling gaps in the dataset.

Through these architectural improvements, the CNN and GAN components can be evolved into production-ready systems capable of high-precision beam diagnostics in a live accelerator environment.

5.2 Customised Graphical User Interface (GUI)

While the current system is operated through Python scripts and Jupyter notebooks, a well-designed GUI can significantly enhance usability, integration, and accessibility for accelerator physicists and beamline operators.

5.2.1 GUI Objectives

The GUI should serve multiple purposes:

- **Image Input & Inference**: Load BPM images in real-time or batch mode and display the predicted beam parameters using the CNN model.

- **Live Visualization**: Display time-series plots of beam position, size, and predicted tilt for live monitoring.
- **Model Interaction**: Allow users to switch between different trained CNN or GAN models, inspect their summaries, and re-trigger evaluations.
- **Synthetic Dataset Generation**: Include sliders or input fields to define beam parameters and trigger GAN-based image generation for dataset expansion.
- **Training Module**: Optionally provide a back-end interface to fine-tune or retrain models on new data without requiring deep ML knowledge.

5.2.2 Technical Stack

To implement this GUI efficiently, several technologies and frameworks can be considered:

- **Frontend**: Python-based libraries like PyQt, Tkinter, or Kivy for native applications; or Dash/Streamlit for browser-based apps.
- **Backend**: TensorFlow/Keras model integration with dynamic loading and evaluation.
- **File I/O**: Numpy-based .npy file parsing, image previews, and real-time directory monitoring.

The GUI can also be extended to include diagnostic tools such as anomaly alerts, beam deviation logging, or configuration export for beamline tuning. Such a tool would transform the project from a research framework into a deployable engineering tool.

5.3 Feedback-Based Smart Control System for Real-Time Monitoring and Calibration

Ultimately, the most powerful application of this project lies in closing the loop between beam diagnostics and control—transforming it into an autonomous feedback system that actively tunes the accelerator components in response to beam conditions.

5.3.1 System Integration

The idea is to link the CNN-based inference engine with control systems governing quadrupole magnets, correctors, or RF cavities. A simple PID-based or ML-based controller can interpret the CNN's predicted beam offsets and trigger control actions to minimize deviations.

This can be achieved through:

- **Mapping Modules**: Train a neural network or inverse model that maps (x,y) deviations to magnet setting corrections.

- **Image-to-Magnet Control Pipeline**: Use synthetic data to simulate how beam parameter changes affect magnet currents and learn a mapping from BPM images → magnet settings.
- **STN-Aware Controllers**: For beam images with rotation/tilt, include a Spatial Transformer Network (STN) that aligns the image before prediction and control.

5.3.2 Challenges and Solutions

- **Latency Constraints**: Real-time diagnostics require rapid inference (< 10 ms). This can be achieved by exporting the trained CNN model using TensorFlow Lite or ONNX and running on edge devices.
- **Safety Mechanisms**: Control actions must be bounded and monitored to prevent damage. Integrating physical constraints or a simulation-based verification layer can reduce risk.
- **Training Data**: A major limitation is the availability of real beam data. GAN-generated augmentation and transfer learning on real samples can bridge this gap.

5.3.3 Potential Outcomes

- **Auto-calibration of BPM Systems**: The CNN-GAN framework can be used to generate controlled deviations and recalibrate the BPM sensors.
- **Anomaly Detection**: Sudden deviations in predicted parameters over time can be flagged as beam anomalies, triggering operator intervention or automated mitigation.
- **Digital Twin**: This system can serve as a digital twin of the actual beamline, useful in testing control strategies, simulating faults, or performing "what-if" scenarios.

Final Remarks

With strategic enhancements to the CNN-GAN architecture, interface design, and integration into control systems, this project has the potential to become a vital part of next-generation synchrotron beam diagnostics and control infrastructure. The fusion of deep learning, synthetic image generation, and feedback systems opens pathways toward smarter, more responsive accelerators, enabling both scientific progress and operational stability.

List of Figures

Fig. 1.1: Building of Indus Accelerator Complex at RRCAT.

Fig. 1.2 – Visual representation of CNN model.

Fig. 1.3 – A Small BPM at Fermilab Accelerator Science and Technology

Fig. 1.4 – BPM Image from Indus 2 Particle Accelerator

Fig. 2.1 – Sample Image from Synthetic BPM Dataset

Fig. 3.1 – GAN Generator Model Summary

Fig. 3.2 – GAN Discriminator Model Summary

Fig. 3.3 – GAN Model Training Hyperparameters Summary

Fig. 3.4 – Loss Plot for GAN Model

Fig. 3.5 – Pixel Value Distribution Plot for GAN Model

Fig. 4.1 – CNN Model Structure Summary

Fig. 4.2 – Evaluation Metrics for CNN Model

Fig. 4.3 – Loss Plots for CNN Model

Fig. 4.4 – Ground Truth (GT) vs Predicted Values (Pred) Plots for CNN Model

Fig. 4.5 – Sample Image for testing the Prediction of CNN Model

Bibliography

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. "Generative Adversarial Nets." *Advances in Neural Information Processing Systems*, 2014.
2. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE*, 1998.
3. Chollet, F. *Deep Learning with Python*, 2nd Edition. Manning Publications, 2021.
4. O'Shea, K., & Nash, R. "An Introduction to Convolutional Neural Networks." *arXiv preprint arXiv:1511.08458*, 2015.
5. Kingma, D.P., & Ba, J. "Adam: A Method for Stochastic Optimization." *arXiv preprint arXiv:1412.6980*, 2014.
6. Mirza, M., & Osindero, S. "Conditional Generative Adversarial Nets." *arXiv preprint arXiv:1411.1784*, 2014.
7. TensorFlow Documentation. https://www.tensorflow.org/
8. Keras API Documentation. https://keras.io/
9. NumPy Documentation. https://numpy.org/doc/
10. Matplotlib Documentation. https://matplotlib.org/
11. INDUS-2 User Guide. Raja Ramanna Centre for Advanced Technology (RRCAT), Indore.
12. Sharma, A., & Pant, H.C. "Beam Diagnostics in Synchrotron Light Sources." *Journal of Instrumentation*, 2020.
13. Bishop, C.M. *Pattern Recognition and Machine Learning.* Springer, 2006.
14. Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising." *IEEE Transactions on Image Processing*, 2017.
15. Radford, A., Metz, L., & Chintala, S. "Unsupervised Representation Learning with Deep Convolutional GANs." *arXiv preprint arXiv:1511.06434*, 2015.
16. Smilkov, D., Thorat, N., Kim, B., et al. "TensorFlow Playground: An Interactive Visualization of Neural Networks." 2016.
17. Van der Walt, S., Colbert, S.C., & Varoquaux, G. "The NumPy Array: A Structure for Efficient Numerical Computation." *Computing in Science & Engineering*, 2011.

18. OpenCV-Python Tutorials Documentation. https://docs.opencv.org/
19. RRCAT Official Website. https://www.rrcat.gov.in/
20. Ioffe, S., & Szegedy, C. "Batch Normalization: Accelerating Deep Network Training." *arXiv preprint arXiv:1502.03167*, 2015.